CodyTolene / **pip-boy-3000-holotapes**

Code | Issues | Pull requests 1 | Actions | Projects | Security and quality | Insights

Watch 0

A repository for Pip-Boy 3000 apps & games created by the community, and hosted on Pip-Boy.com

www.pip-boy.com

MIT license

Contributing

8 stars **13** forks **0** watching Branches Activity
Tags

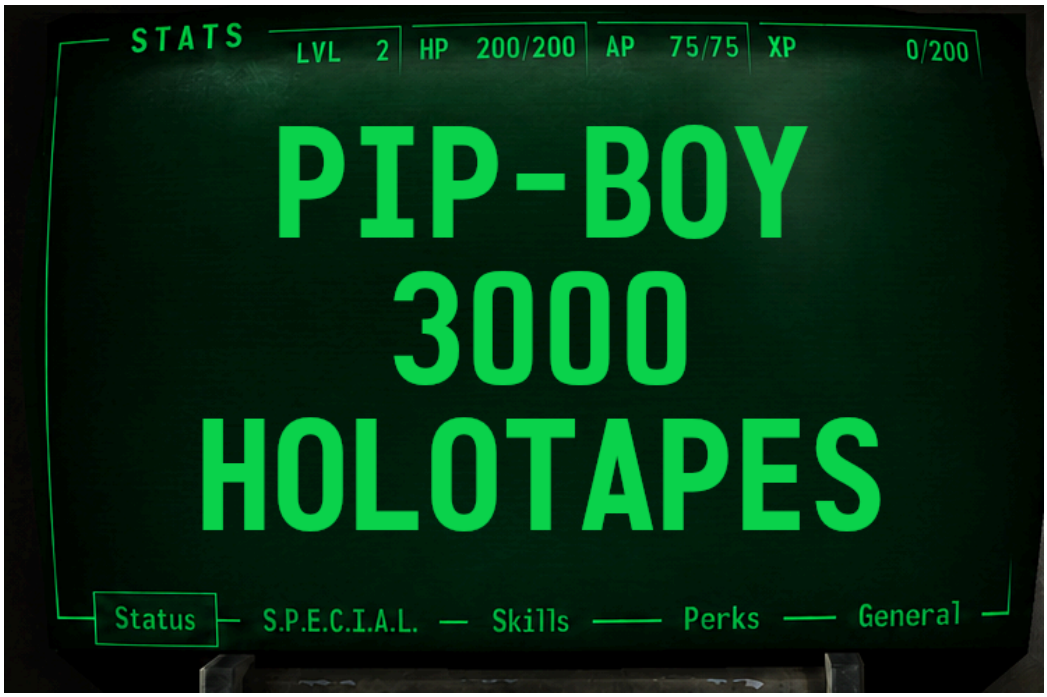
Public repository

2 Branches **0** Tags Go to file T Go to file Add file Code

CodyTolene Reorganize preview images, menu first 5b1add6 · 17 hours ago

.github	Update refs	2 months ago
.husky	Add build scripts w/husky	2 months ago
.scripts	Fix up registry build	yesterday
.vscode	Add build scripts w/husky	2 months ago
holotapes	Reorganize preview images, menu f…	17 hours ago
.gitignore	Add build scripts w/husky	2 months ago
.prettierignore	Add build scripts w/husky	2 months ago
CODEOWNERS	iPip (v2.0.0) > RobCo Entertainment…	3 days ago
CONTRIBUTING.md	Documentation Updates (#47)	last week
LICENSE	Documentation Updates (#47)	last week
README.md	Documentation Updates Rev 2 (#49)	last week
agents.md	Documentation Updates Rev 2 (#49)	last week
package-lock.json	Update refs	2 months ago
package.json	Update refs	2 months ago
prettier.config.cjs	Add build scripts w/husky	2 months ago
	Add build scripts w/husky	2 months ago

tsconfig.json



Pip-Boy 3000 Holotapes

A community driven repository of custom applications and games for the Pip-Boy 3000, hosted on pip-boy.com.

Pip-Boy.com | Discord Community | Bethesda Store | The Wand Company | Espruino | RobCo Industries

Index

Description

Pip-Boy 3000 Holotapes by the community, for the community.

Install on: pip-boy.com

Follow the guide below to create your own custom Holotapes for the Pip-Boy 3000!

[Index]

Creating a new Holotape

1. Create a directory for the app or game under `holotapes/`. Every Holotape must use this structure:

```
holotapes/<YourHolotape>/
├── app.js
├── app.min.js
├── metadata.json
├── README.md
├── ChangeLog
└── assets/
```

2. Write the unminified source in `app.js`. The app must be an anonymous function expression that the Pip-Boy OS invokes; do not invoke it yourself with a trailing `()`. It must return an uppercase alphanumeric `id` and a `remove` function.

   Example app:

   ▶ Expand/Collapse

3. Add `metadata.json`. Asset paths and storage URLs are relative to the Holotape directory. The storage directory is the metadata `id` converted to uppercase, with underscores replacing hyphens.

```
{
  "id": "example",
  "name": "Example Holotape",
  "author": "@your-github-username",
  "version": "1.0.0",
  "description": "A short, one-sentence description.",
  "icon": "assets/icon.png",
  "previews": [],
  "type": "app",
  "readme": "README.md",
  "storage": [{ "name": "HOLO/EXAMPLE/APP.JS", "url": "app.min.js" }],
  "storageOptional": []
}
```

   The metadata `id` must contain only lowercase letters, numbers, and hyphens; `version` must use semantic versioning; and `type` must be exactly `app` or `game`. Do not edit `holotapes/registry.json` manually. `npm run build` generates it from each `metadata.json` file.

4. Document the description, controls, installation, tested firmware, and credits in the Holotape's `README.md`.

5. Add at least one `ChangeLog` entry in this format:

```
1.0.0 (yyyy-mm-dd)
<pull-request-or-change-link>
- Initial release
```

6. Generate `app.min.js` from `app.js` as described in Build and minification. Both files must behave identically.

[Index]

Development Workflow

Using Pip-Boy.com

▶ Expand/Collapse

Using the Espruino Web IDE

You can use one of the two methods below to upload and test your Holotape:

▶ Expand/Collapse

Build and minification

Minify and Encode your Holotape here:

https://www.pip-boy.com/3000/holotapes/create

This will give you a proper `app.min.js` from `app.js` .

Run the repository build after adding or changing metadata:

```
npm install
npm run build
```

This rebuilds `holotapes/registry.json` , rewrites relative file paths for the registry, and rejects metadata whose `type` is not `app` or `game` . It does not generate `app.min.js` or perform all of the submission checks for you.

[Index]

Images

▶ Expand/Collapse

[Index]

Input handling

▶ Expand/Collapse

[Index]

Memory and Performance

▼ Expand/Collapse

Main rules:

- The display is always 480×320. If dimensions are needed repeatedly, declare them once as `const W =`

never `var`, and inline single-use values.

- Always use the global `h` graphics object directly and chain graphics calls. Do not spend a variable block on an alias for `h`.
- Use `h.clearRect()`, clipping, cached wrapped text, and dirty flags to redraw only changed regions.
- Timer-driven apps normally rely on the OS auto-flush. Do not call `h.flip()` from a `setInterval()` loop. For `Pip.onFrame` rendering, set `Pip.lastFlip = getTime()` before drawing and call `h.flip()` after drawing.
- Use `"ram"` for performance-critical frame functions and `"jit"` for small, numeric tight loops. Do not use either without a measured need.
- Use `Math.randInt(n)` instead of `Math.random()`, and use typed arrays for dense numeric data.
- Espruino does not support `async` / `await`, ES modules, template literals, `fetch()`, `XMLHttpRequest`, or `requestAnimationFrame()`.
- Keep the app scoped:

```
(function () {
  // App code here
  return {
    id: 'APPID',
    notDefault: true,
    fullscreen: true,
    remove: function () {
      /* clean up app resources */
    },
  };
});
```

- Clean up every resource created by the app in `remove()`:

```
  remove: function() {
    Pip.removeListener("knob1", onKnob1);
    clearInterval(intervalId);
    clearTimeout(timeoutId);
    clearWatch(watchId);
    Pip.audioStop();
    h.clear();
  }
```

- Never call `load()` or `E.reboot()` from `remove()`, call `Pip.remove()` at the top of the app, use a bare `clearWatch()`, or delete/reassign OS globals.

Useful memory checks:

```
process.memory();
print(E.getSizeOf(this, 1).sort((a, b) => a.size - b.size));
print(E.getSizeOf(Pip, 1).sort((a, b) => a.size - b.size));
print(E.getSizeOf(this['\xFF'], 1).sort((a, b) => a.size - b.size));
```

`this['\xFF']` shows timers, watches, and internal runtime state.

`Pip.CURRENT` can hold the current page or app code.

[Index]

Contributing

▶ Expand/Collapse

[Index]

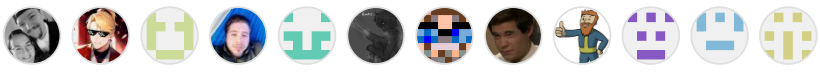
License

This repository is licensed under the MIT License.

All code, holotapes, apps, games, scripts, metadata, documentation, and other contributions submitted to this repository must be licensed under the MIT License unless explicitly stated otherwise by the repository maintainer in writing.

By submitting a pull request or contribution, you agree that your contribution is provided under the MIT License. See CONTRIBUTING.md for details.

Contributors 12



Languages